14 haftalık yol haritasında 8. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 08

Fonksiyonlar I: Modüler Programlama ve Soyutlama

Prototip, tanım ve çağrı; değerle aktarım; fonksiyon sözleşmesi ve kara kutu; standart kütüphane; yukarıdan aşağıya tasarım ve assert ile birim testi

MODÜL IV · MODÜLER PROGRAMLAMA · H8–H9

HAFTANIN BÜYÜK SORUSU

Milyonlarca satırlık yazılımlar, hiçbir mühendis tamamını aklında tutamadığı hâlde nasıl geliştirilir ve doğrulanır?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 3
1.1 Öğrenme çıktıları 3
2. Geçen Haftadan: İç İçe Döngülerden Fonksiyonlara 3
3. Kavramsal Anlatım 4
3.1 Neden fonksiyon? Soyutlama, yeniden kullanım, okunabilirlik, test 4
3.2 Fonksiyonun anatomisi: ilk fonksiyonumuz 4
3.3 Prototip, tanım ve çağrı; başlık dosyalarının rolü 6
3.4 void'in iki anlamı: değer döndürmeyen ve parametre almayan fonksiyonlar 7
3.5 Argüman–parametre eşleşmesi ve değerle aktarım 7
3.6 Kara kutu ve fonksiyon sözleşmesi 8
3.7 Standart kütüphane: belgeden okuyarak kullanmak 9
3.8 Yukarıdan aşağıya tasarım ve yapı şeması 10
3.9 assert ile birim testi 11
3.10 Kaputun altında: bir çağrıda neler olur? 11
4. Çözümlü Örnekler: Örnek Merdiveni 13
4.1 ★ basamaklar: ilk fonksiyonlar (Örnek 1–4) 13
4.2 Örnek 5 ★★ — H6 kodunu fonksiyona taşımak: faktoriyel(n) ve us_al(taban, us) 14
4.3 Amiral örnek — Örnek 6 ★★: is_prime(n), H7 kodunun yeniden düzenlenmesi 14
4.4 Fonksiyonlar birlikte çalışır (Örnek 7–9) 16
4.5 Değerle aktarımın sınırı ve dizisiz sayma (Örnek 10–11) 17
4.6 Örnek 12 ★★★ — assert ile is_prime ve ebob testleri 18
4.7 Amiral örnek — Örnek 13 ★★★: ATM programının fonksiyonlarla yeniden yapılandırılması 19
5. Hata Avcısı 21
6. Mühendis Gözüyle: Modülerlik, Sözleşme ve Test 23
7. Mini-OBS v0.8: Fonksiyonlara Ayrıştırma ve assert Testleri 23
8. Alıştırmalar ve Atölye 24
9. Kendini Değerlendirme 28
10. AP-2 Köprüsü ve Merak Kutusu 28
11. Özet, Kavram Haritası ve Terim Sözlüğü 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Milyonlarca satırlık yazılımlar, hiçbir mühendis tamamını aklında tutamadığı hâlde nasıl geliştirilir ve doğrulanır?

H7'nin ATM programı (`H07_14_atm_menu.c`) 144 satırdı: PIN denetimi, menü, para yatırma ve çekme ile dört kez tekrarlanan girdi doğrulama bloğu tek bir `main`'in içindeydi. Gerçek yazılımlar ise milyonlarca satırdır. Bu haftanın yanıtı **fonksiyonlardır**: problemi, adı konmuş, tek iş yapan ve **ayrı ayrı test edilebilen** parçalara bölmek.

> GERÇEK DÜNYADAN · **Apollo 11, 20 Temmuz 1969: "Program alarm... 1202"**
>
> Ay'a inişin son dakikalarında ay modülünün kılavuzluk bilgisayarı **1202** ve **1201** alarmları verdi; Görev Kontrol inişin sürmesine onay verdi ve Kartal (Eagle) son alarmdan yaklaşık üç dakika sonra Ay'a kondu (Jones & Glover, t.y.). MIT Instrumentation Laboratory programcısı Peter Adler'e göre neden, radar anahtarlarının yanlış yapılandırılması yüzünden var olmayan **buluşma radarı** (rendezvous radar) verilerini işleyecek işlerin tekrar tekrar zamanlanması ve işlere ayrılan sınırlı bellek alanlarının dolmasıydı: 1202 boş "core set", 1201 boş "VAC alanı" kalmadığını bildiriyordu (Adler, 1998).
>
> Yazılım aşırı yükü öngörecek biçimde tasarlanmıştı: bilgisayar kendini yeniden başlattı, seçili programları kaldıkları yere yakın bir noktadan yeniden kurdu ve hatalı radar işlerini kurmadı (Adler, 1998); aşırı yükte **en yüksek öncelikli işler sürdü**, kuyruktan düşenler en düşük öncelikli işlerdi (Martin, 1994). Uçuş yazılımı çalışmasını, MIT Instrumentation Laboratory'de Yazılım Mühendisliği Bölümü'nü (Software Engineering Division) yöneten Margaret Hamilton yürütüyordu; "yazılım mühendisliği" kavramını yaygınlaştırdığı kabul edilir (Computer History Museum, t.y.; NASA Science, 2024).
>
> Bağlantı: sistem, her biri belirli bir işten sorumlu ve önceliği tanımlı **iş parçalarından** kuruluydu. Bir fonksiyon da adı, girdisi (parametreleri), çıktısı (dönüş değeri) ve sözleşmesi tanımlı bir iş parçasıdır (olayın kavramsal gösterimi sunumda ve web sayfasındadır).

1.1 Öğrenme çıktıları

ÖÇ 8.1 Fonksiyon tanımı, prototipi ve çağrısı sözdizimini doğru kullanır. *(Bölüm 3.2–3.4, Örnek 1–3)*

ÖÇ 8.2 Değerle aktarım (pass by value) mekanizmasını açıklar ve sonuçlarını öngörür. *(Bölüm 3.5, Örnek 10)*

ÖÇ 8.3 Bir problemi yukarıdan aşağıya adım adım iyileştirme (stepwise refinement; Wirth, 1971) ile alt görevlere ayırır ve her birini fonksiyon olarak gerçekler. *(Bölüm 3.8, Örnek 13)*

ÖÇ 8.4 Standart kütüphane fonksiyonlarını (math.h, ctype.h, stdlib.h, time.h) belgelerinden okuyarak kullanır. *(Bölüm 3.7, Örnek 11)*

ÖÇ 8.5 assert ile fonksiyonlar için basit birim testleri yazar. *(Bölüm 3.9, Örnek 12)*

2. Geçen Haftadan: İç İçe Döngülerden Fonksiyonlara

H7'de iç içe döngüler, asallık testinin √n ile hızlandırılması ve menü döngüsü vardı. Üç geri çağırma sorusu (programlar `kodlar/quiz/` klasöründe):

> TAHMİN ET · **Geri çağırma: üç soru**
>
> - **S1 (iç içe döngü).** `H08_Q01_ic_ice_sayac.c`: `i = 1..4`, `j = i..4`; `sayac++` kaç kez çalışır?
> - **S2 (√n asallık).** `H08_Q02_karekok_bolen.c`: 97 için iyileştirilmiş test (tek bölenler, d ≤ n / d) kaç bölen dener; naif test kaç?
> - **S3 (menü döngüsü).** `H08_Q03_menu_break.c` girdi `1 2 1 3` ile ne yazdırır; `switch` içindeki `break` neden döngüyü bitirmez?

Yanıtlar (gerçek çıktılar). S1: `10` = 4 + 3 + 2 + 1 (n(n+1)/2 sezgisi). S2: "denenen bölen: 4, asal mı: 1": d = 3, 5, 7, 9; 11 > 97 / 11 = 8 olduğundan döngü biter. Naif test 2..96 aralığındaki **95** böleni denerdi. S3: "ABA bitti": `break` yalnızca `switch`'ten çıkar. H7'nin asallık testi (`H07_06_asal_naif.c`, `H07_11_asal_karekok_sure.c`) Örnek 6'da `is_prime()`'a, `H07_14_atm_menu.c` Örnek 13'te fonksiyonlara bölünmüş bir programa dönüşecek.

3. Kavramsal Anlatım

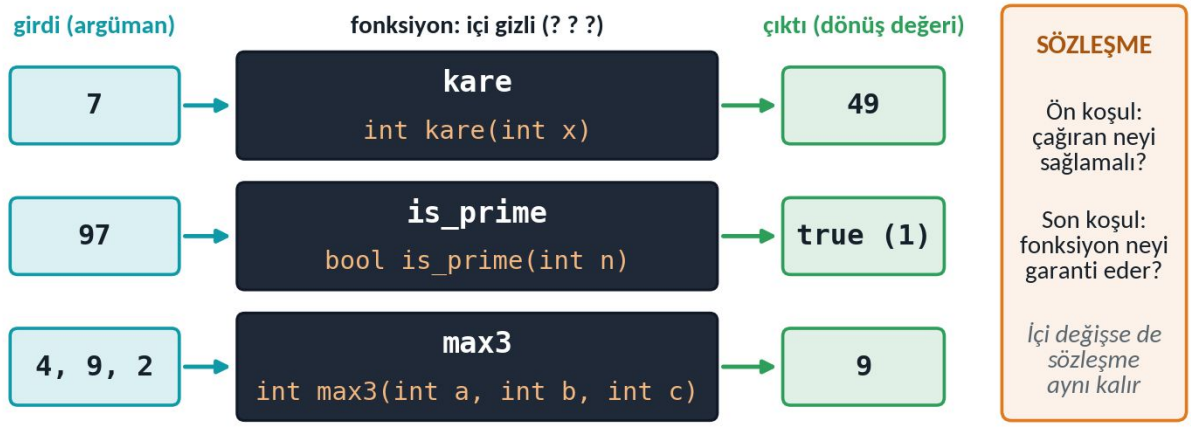
3.1 Neden fonksiyon? Soyutlama, yeniden kullanım, okunabilirlik, test

Bir **fonksiyon** (function), adı verilmiş, belirli bir işi yapan, gerektiğinde **parametre** alıp bir **dönüş değeri** (return value) üreten kod bloğudur; `main`, `printf` ve `scanf` de birer fonksiyondur. Dört kazancı vardır: **soyutlama** (abstraction: `is_prime(n)`'i çağıran, asallığın nasıl değil yalnızca **ne** yapıldığını bilir), **yeniden kullanım** (reuse: bir kez yaz, her yerden çağır; hata tek yerde düzelir, DRY), **okunabilirlik** (`if (is_leap_year(yil))` bir cümle gibi okunur) ve **test edilebilirlik** (küçük fonksiyon, programdan bağımsız denenir; Bölüm 3.9).

`H08_K01_tekrar_fonksiyon.c` üç ortalamayı önce `0.40 * 70 + 0.60 * 85` formülünü üç kez yazarak, sonra `agirlikli_ortalama(vize, final)`'ı üç kez çağırarak hesaplar; çıktılar aynıdır ("Ali : 79.00 · Ayşe : 93.00 · Can : 46.80"). Ama ağırlıklar değişirse ilkinde üç satır, ikincisinde tek satır düzeltilir.

Fonksiyon bir **kara kutudur** (black box; Şekil 8.1): argümanlar girer, dönüş değeri çıkar, içi gizlidir; çağıranın güvendiği tek şey fonksiyonun **sözleşmesidir** (Bölüm 3.6).

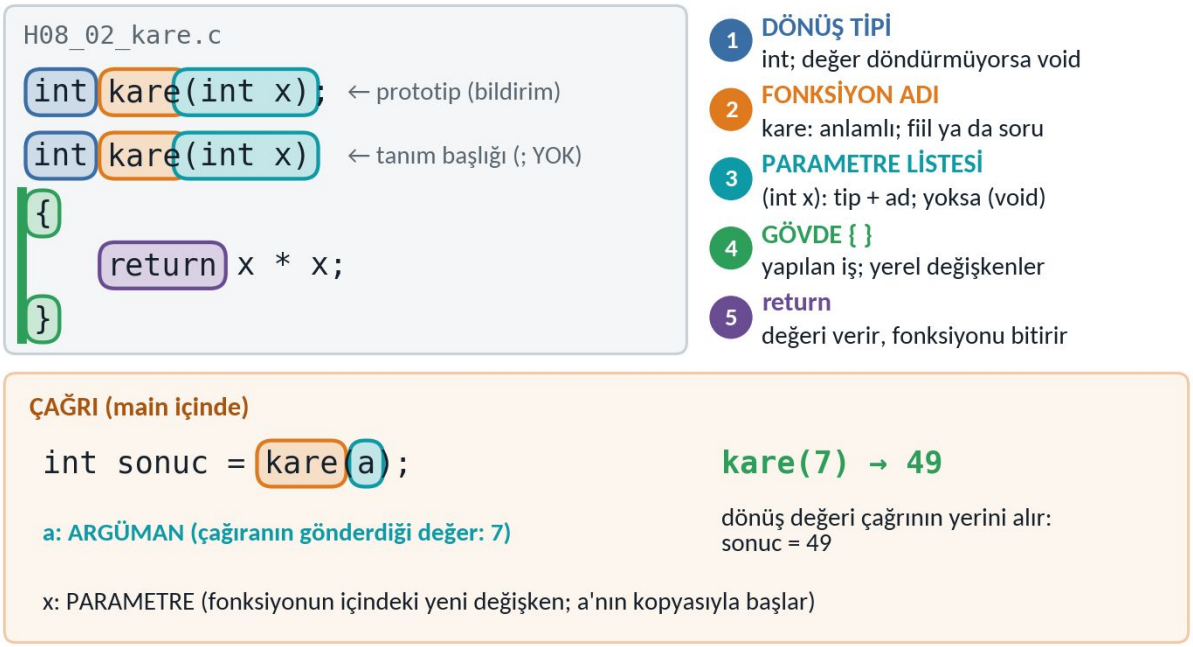
Şekil 8.1
Fonksiyon = kara kutu: kare, is_prime ve max3 için girdi–çıktı ve sözleşme



3.2 Fonksiyonun anatomisi: ilk fonksiyonumuz

Bir fonksiyon **tanımı** (definition), **başlık** (dönüş tipi, ad, parametre listesi) ve süslü parantezler arasındaki **gövdeden** oluşur (Şekil 8.2): `dönüş_tipi fonksiyon_adı(tip1 parametre1, …) { deyimler; return ifade; }`.

Şekil 8.2
Fonksiyonun anatomisi: renk kodlu parçalar, prototip, tanım ve çağrı (H08_02_kare.c)



En yalın fonksiyon hiçbir şey almaz, hiçbir şey döndürmez. Örnek 1, karşılama iletisini `selamla()`'ya koyar ve iki kez çağırır:

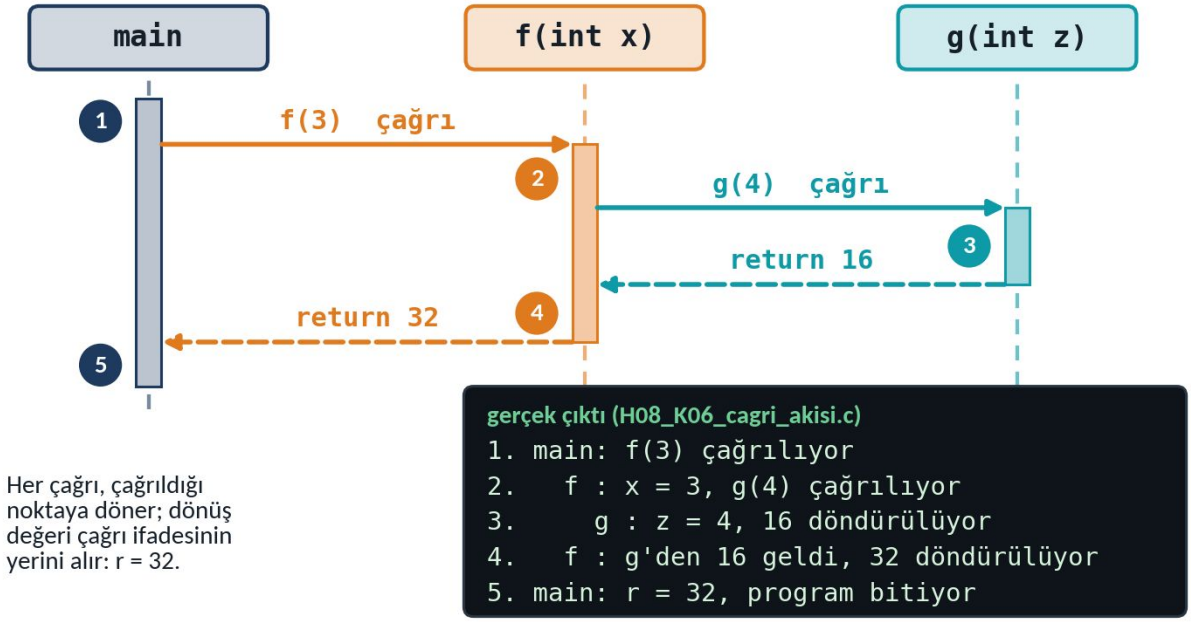
H08_01_selamla.c · satır 8–26

```
 8  #include <stdio.h>
 9
10  void selamla(void);                    /* prototip: "böyle bir fonksiyon" */
11
12  int main(void)
13  {
14      printf("main başladı.\n");
15      selamla();                         /* çağrı: akış selamla'ya gider... */
16      selamla();                         /* ...ve her seferinde buraya döner */
17      printf("main bitti.\n");
18      return 0;
19  }
20
21  /* Ekrana iki satırlık bir karşılama iletisi yazar. */
22  void selamla(void)                     /* tanım: başlık + gövde */
23  {
24      printf("  Merhaba! AP-1'e hoş geldiniz.\n");
25      printf("  Bu hafta: Fonksiyonlar I\n");
26  }
```

Satır 10: **prototip** (bildirim): "`selamla` adında, parametre almayan, değer döndürmeyen bir fonksiyon var." Satır 15: **çağrı** (call): yürütme gövdeye atlar, gövde bitince çağrının arkasından sürer (gerçek çıktı: "main başladı.", iki karşılama, "main bitti."). Satır 22–26: **tanım**. Adlar snake_case ve ASCII Türkçedir: iş yapana fiil (`selamla`), değer hesaplayana ad (`kare`), evet/hayır sorusuna soru (`gecti_mi`).

İç içe çağrıda kural aynıdır: `H08_K06_cagri_akisi.c`'de `main` `f(3)`'ü, `f` de `g(4)`'ü çağırır; her `return` akışı çağrının yapıldığı noktaya geri götürür ve dönüş değeri çağrı ifadesinin yerini alır (Şekil 8.3).

Şekil 8.3
Çağrı akışı: main → f → g ve dönüşler (H08_K06_cagri_akisi.c'nin gerçek çıktısıyla)



Her çağrı, çağrıldığı noktaya döner; dönüş değeri çağrı ifadesinin yerini alır: r = 32.

Parametre ve dönüş değeri

Örnek 2'de `kare` bir `int` alıp bir `int` döndürür. `return x * x;` değeri çağırana **verir** ve fonksiyonu **bitirir**; `int sonuc = kare(a);` satırı `int sonuc = 49;` gibi davranır (kod Şekil 8.2'de):

Argüman bir sabit, değişken, ifade (`kare(a + 1)`: önce 8 hesaplanır) ya da başka bir çağrı (`kare(kare(3))`: önce içteki 9 döner) olabilir; gerçek çıktı: "kare(7) = 49", "kare(5) = 25", "kare(a + 1) = 64", "kare(kare(3)) = 81". `kare` $|x| \le 46\,340$ için doğrudur; $46\,341^2$ `INT_MAX`'ı aşar (işaretli taşma tanımsız davranıştır).

3.3 Prototip, tanım ve çağrı; başlık dosyalarının rolü

Derleyici dosyayı **yukarıdan aşağıya** okur; bir çağrıyı gördüğü anda fonksiyonun dönüş ve parametre tiplerini bilmelidir (argümanları dönüştürmek, sayılarını denetlemek, dönüş değerini yorumlamak için). İki yol vardır: tanımı çağrıdan **önce** yazmak (Alıştırma 8.2) ya da başa bir **prototip** koymak. Ders kodlarında ikincisini seçiyoruz: dosya sırasıyla `#include`, prototipler, `main` ve tanımlardan oluşur; `main` programın "içindekiler" sayfası gibi okunur.

Prototip, sonunda noktalı virgül olan bir başlıktır: `int kare(int x);` (parametre adları isteğe bağlıdır). Prototip ile tanımın **imzası** (dönüş tipi, ad, parametre tipleri ve sayısı) aynı olmalıdır. **Başlık dosyaları** (header files) prototip koleksiyonlarıdır: `#include <stdio.h>` `printf`'in prototipini getirir; **tanımı** derlenmiş C kütüphanesindedir (glibc) ve bağlayıcı ekler (H1). Prototip unutulursa ne olacağını Bölüm 5'te (HA1) göreceğiz: GCC 13 uyarır, Clang 16+ ve GCC 14+ derlemez (GCC Team, 2024; LLVM Project, 2023).

3.4 void'in iki anlamı: değer döndürmeyen ve parametre almayan fonksiyonlar

`void` **dönüş tipinde** "değer döndürmez" (gövdede yalın `return;` yazılabilir), **parametre listesinde** `(void)` "parametre almaz" demektir:

H08_K03_void_ornekleri.c · satır 23–32

```c
void cizgi(int uzunluk)
{
    if (uzunluk <= 0) {
        return;                          /* değer yok: yalın return */
    }
    for (int i = 0; i < uzunluk; i++) {
        printf("-");
    }
    printf("\n");
}
```

`cizgi` parametre alır ama değer döndürmez (`cizgi(0)` hiçbir şey çizmez); aynı dosyadaki `hafta_no(void)` ise tersidir: gerçek çıktı "AP-1 · Hafta 8". `void` fonksiyonun sonucu atanamaz: `int x = yaz(3);` derleme hatasıdır ("void value not ignored as it ought to be", `H08_Q14_void_deger.c`).

Neden () değil de (void)?

C17'de `int bugun();` bir **prototip değildir**; "parametreleri belirtilmemiş" demektir. `H08_K04_bos_parantez.c`'de böyle bildirilen sıfır parametreli fonksiyon `bugun(1, 2)` ile iki argümanla çağrıldığı hâlde GCC 13.3 `-std=c17 -Wall -Wextra -pedantic` ile **hiç uyarı vermedi** ve program "bugun(1, 2) = 8" yazdı. Clang 18 yalnızca "deprecated" uyarısı verir; C23'te `()` artık `(void)` demektir ve `gcc -std=c2x` aynı dosyada "error: too many arguments to function 'bugun'" verir. **Kural:** parametresiz fonksiyonu her zaman `(void)` ile bildirin (`int main(void)` gibi).

3.5 Argüman–parametre eşleşmesi ve değerle aktarım

Çağrıdaki değerlere **argüman** (argument), tanımdaki değişkenlere **parametre** (parameter) denir; sırayla eşleşirler, sayıları eşit olmalıdır ve argümanlar gerekirse prototipteki tipe **dönüştürülür** (HA4).

C'de argümanlar **değerle aktarılır** (pass by value): her argümanın değeri hesaplanır ve parametreye **kopyalanır**. Parametre fonksiyonun yerel değişkenidir; değişmesi çağırana yansımaz (Şekil 8.4):

H08_K05_deger_aktarim.c · satır 12–27

```c
int main(void)
{
    int sayi = 5;
    printf("main : önce  sayi = %d\n", sayi);
    artir(sayi);
    printf("main : sonra sayi = %d   (değişmedi)\n", sayi);
    artir(sayi + 10);                    /* önce 15 hesaplanır */
    return 0;
}

void artir(int x)
{
    printf("artir: gelen x = %d\n", x);
    x = x + 1;
    printf("artir: x = %d   (yalnızca kopya değişti)\n", x);
}
```

Şekil 8.4
Değerle aktarım: sayi ile x iki ayrı bellek kutusudur; x'in değişmesi sayi'yi etkilemez



Gerçek çıktıda `artir` içinde x 6 olur, ama `main`'e dönünce "sayi = 5 (değişmedi)" yazılır; `artir(sayi + 10)`'da önce 15 hesaplanıp kopyalanır. **Güvenlik:** fonksiyon çağıranın değişkenlerini bozamaz (Örnek 7'de `ebob` parametrelerini değiştirir, `main`'deki `a`, `b` aynı kalır). **Sınır:** çağıranın değişkenini **değiştirmek** gerekirse? Örnek 10'daki `takas(a, b)` bu bilinçli açık sorudur (çözüm H12'de).

Merak: argümanlar hangi sırayla hesaplanır? `f(g(), h())`'de önce hangisinin çalışacağını C standardı **belirtmez** (unspecified); yalnızca hepsinin fonksiyona girmeden önce hesaplanacağını garanti eder (ISO/IEC, 2018). İç içe çağrıda sıra kesindir: `f(g(1))`'de önce `g(1)` çalışır (Alıştırma 8.2).

3.6 Kara kutu ve fonksiyon sözleşmesi

Fonksiyonu kullanmak için **sözleşmesini** (contract) bilmek yeter. Başa yorum olarak yazılır:

- **Ön koşul** (precondition): çağıranın sağlaması gerekenler (`ebob(a, b)` için a, b > 0).
- **Son koşul** (postcondition): ön koşul sağlandıysa fonksiyonun garantisi ("n asalsa true döndürür").

Ön koşul ihlal edilirse garanti yoktur (`faktoriyel(n)` için 0 ≤ n ≤ 20; `faktoriyel(21)` taşar, H06_05). Sözleşmenin gücü, kutunun içini değiştirme özgürlüğüdür; `H08_K02_kara_kutu.c` aynı sözleşmeyi iki gerçekleştirimle karşılar:

H08_K02_kara_kutu.c · satır 24–35

```c
/* SÖZLEŞME (iki gerçekleştirim için de aynı)
 * Ön koşul : 0 <= n <= 100000
 * Son koşul: 1 + 2 + ... + n toplamını döndürür (n = 0 için 0).
 * Kara kutu 1: n tur süren döngü (H6, Örnek 2). */
long long toplam_dongu(int n)
{
    long long toplam = 0;
    for (int i = 1; i <= n; i++) {
        toplam += i;
    }
    return toplam;
}
```

İkinci gerçekleştirim tek satırdır: `return (long long)n * (n + 1) / 2;` (Gauss, H2). Biri n tur döner, öbürü sabit sayıda işlem yapar; gerçek çıktıda n = 0, 10, 100 ve 100 000 için ikisi aynı değerleri verir (son satır 5 000 050 000): çağıran için ikisi **aynıdır**. Toplam `int`'e sığmadığından dönüş tipi `long long`'dur. Sözleşme, "tam olarak ne yapmalı?" sorusunu kodu yazmadan yanıtlatır ve testleri belirler (Bölüm 3.9).

bool döndüren doğrulama ve bileşim. Evet/hayır sorusu soran fonksiyonlar `<stdbool.h>`'deki `bool`'u döndürür ve soru gibi adlandırılır (`is_even`, `is_leap_year`, `is_prime`, `gecti_mi`); gövdeleri çoğu zaman tek bir mantıksal ifadedir (`return n % 2 == 0;`). Asıl güç **bileşimdedir** (composition): `max3`, işi `max2`'ye devreder: `return max2(max2(a, b), c);` (Örnek 3); H5'teki `H05_04_uc_sayi_en_buyuk.c` aynı işi on satırlık iç içe `if` ile yapıyordu.

3.7 Standart kütüphane: belgeden okuyarak kullanmak

C standardı bir **standart kütüphane** tanımlar (ISO/IEC, 2018). Bir fonksiyonu belgesinden dört bilgiyle okuruz: **başlık** (`#include`), **imza**, **anlam** ve **özel durumlar**; Linux'ta `man 3 sqrt`. Tablo 8.1 bu haftanın sık kullanılan fonksiyonlarını verir.

Tablo 8.1
Bu haftanın standart kütüphane fonksiyonları (imzalar C standardından sadeleştirilmiştir)

Başlık	İmza (sadeleştirilmiş)	Döndürdüğü / yaptığı	Dikkat
math.h	`double sqrt(double x) · pow(double x, double y)`	$\sqrt{x}$ · x^y	x < 0: nan; Linux'ta -lm
math.h	`double fabs(double x) · floor(x) · ceil(x)`	|x| · aşağı / yukarı yuvarlama	floor(-2.5) = -3
ctype.h	`int isdigit(int c) · isalpha · isspace`	doğruysa sıfırdan farklı int	c: unsigned char ya da EOF
ctype.h	`int toupper(int c) · tolower(int c)`	dönüştürülmüş karakter	dönüş değerini atayın
stdlib.h	`int abs(int n)`	|n|	abs(INT_MIN) tanımsız
stdlib.h	`int rand(void) · void srand(unsigned s)`	0..RAND_MAX sözde rastgele	srand bir kez
stdlib.h	`void exit(int durum)`	programı sonlandırır	EXIT_SUCCESS / FAILURE
time.h	`time_t time(time_t *t)`	takvim zamanı	time(NULL) yeterli

<math.h> ve -lm

`H08_K08_math.c` okunan bir sayıyla `sqrt`, `pow`, `fabs`, `floor`, `ceil` ve kendi `hipotenus(a, b)` fonksiyonunu (`return sqrt(a * a + b * b);`) dener. −2.5 girildiğinde (gerçek çıktı) iki tuzak görünür: `sqrt(-2.50)` için glibc `-nan` yazar (gerçel sayılarda tanımsız); `floor(-2.50)` = −3, `ceil(-2.50)` = −2, çünkü `floor` −∞ yönüne yuvarlar (sıfıra doğru kırpan `(int)`'ten farklı, H4).

Linux'ta (glibc) matematik fonksiyonları ayrı bir kütüphanededir (`libm`); bağlarken `-lm` gerekir. Unutulursa bağlayıcı "undefined reference to 'sqrt'" der (`H08_K10_lm_degisken.c`). Ama `H08_K09_lm_sabit.c` `sqrt(2.0)` ve `pow(2.0, 10.0)` çağırdığı hâlde `-lm`'siz de **derlendi**: argüman sabitse GCC değeri derlemede hesaplar; assembly'de `call sqrt` sayısı 0. Sabitle yapılan deneme unutulan `-lm`'yi **gizler**: `math.h` kullanan her programı `-lm` ile derleyin (MinGW ve macOS'ta gerekmez, zararı yoktur).

<ctype.h>: karakter sınıfları

`H08_K11_ctype.c` bir satırı `getchar` ile okuyup `isdigit`, `isalpha`, `isspace` ile sayar, `toupper` ile yazar. (1) `isdigit('7')` bu makinede **2048** döndürdü: "doğru" 1 değil, **sıfırdan farklı herhangi bir int**tir; sonucu `== 1` ile karşılaştırmayın. (2) Argüman `unsigned char` aralığında ya da `EOF` olmalıdır; aksi tanımsız davranıştır (ISO/IEC, 2018). `char` ile çağırırken `isdigit((unsigned char)ch)` yazın. UTF-8'de Ç, ğ, ı iki baytlıdır; ctype onları harf saymaz: "Çağrı 2026" girdisi "ÇAğRı 2026" ve "rakam: 4, harf: 2" verdi (gerçek çıktı; H13).

<stdlib.h> ve <time.h>

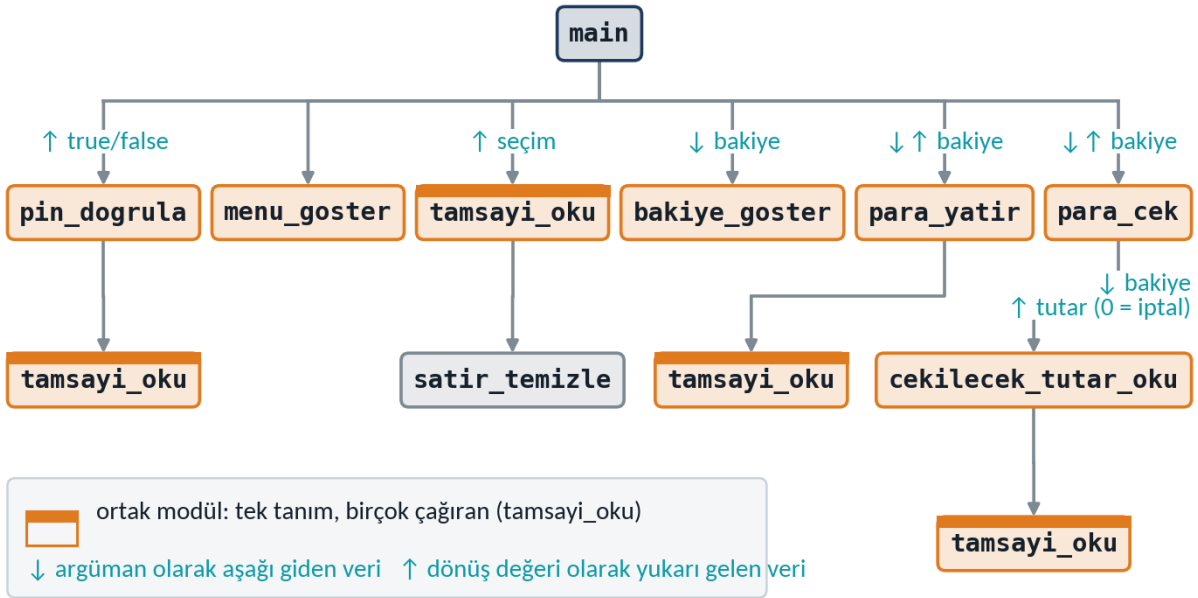
`H08_K12_stdlib.c`'deki `int bol(int a, int b)`'nin ön koşulu `b != 0`'dır; ihlalde ileti yazıp `exit(EXIT_FAILURE);` çağırır. `exit` programı **bulunduğu fonksiyondan** bitirir: `bol` `main`'e dönmez, son satır yazılmaz, çıkış kodu 1 olur (gerçek çalıştırma, b = 0). `main`'deki `return n;` ile `exit(n);` eşdeğerdir.

`time(NULL)` (`<time.h>`) takvim zamanını döndürür; Linux'ta (POSIX) 1 Ocak 1970 00:00 UTC'den bu yana geçen saniyedir (`H08_K13_time.c`: "time(NULL) = 1790836264 saniye"); H6'daki `srand((unsigned)time(NULL))` tohumu buradan gelir. `H08_K14_rand_aralik.c`'deki `rastgele(alt, ust)` ise `return alt + rand() % (ust - alt + 1);` ile zar ya da not üretir.

3.8 Yukarıdan aşağıya tasarım ve yapı şeması

Programı nereden bölmeli? Wirth'in **adım adım iyileştirme** (stepwise refinement) yöntemi: problemi tek cümleyle yaz; her adımda bir kutuyu **daha ayrıntılı alt görevlere** ayır; alt görev doğrudan koda dönüşecek kadar basitleşince dur (Wirth, 1971). Her kutu bir fonksiyon adayıdır. ATM için: Adım 0 "ATM'yi çalıştır"; Adım 1 "PIN'i doğrula → menü döngüsü (seç, işlem yap) → çık"; Adım 2 "bakiyeyi göster / para yatır / para çek"; Adım 3 "para çek: 10'un katı, bakiyeyi aşmayan tutarı oku (0 = iptal), bakiyeden düş". Varılan fonksiyonlar için bkz. Şekil 8.5.

Şekil 8.5
ATM programının yapı şeması (H08_13_atm_fonksiyonlu.c): çağrılar ve veri akışı



İlk adım hemen koda dökülür: `main` yalnızca alt görev adlarıyla yazılır, alt görevler adını yazıp sabit değer döndüren **koçan** (stub) fonksiyonlardır; program derlenir ve çalışır. `H08_K17_iskelet_atm.c`'nin `main`'i `pin_dogrula()`, `menu_secimi()` ve `bakiye =`

`islem_yap(secim, bakiye);` çağrılarından ibarettir; her koçan "[koçan] pin_dogrula: PIN doğru sayıldı" gibi bir satır yazar ve program "Son bakiye: 1000 TL" ile biter (gerçek çıktı).

Koçanlar sonra birer birer doldurulur ve ayrı ayrı denenir. Fonksiyonlar arasındaki **çağırma ilişkisi yapı şemasıyla** (structure chart) çizilir (Şekil 8.5): kutu = fonksiyon, çizgi = çağrı; küçük oklar aşağı inen (argüman) ve yukarı çıkan (dönüş değeri) veriyi gösterir. Akış diyagramından farkı: **ne zaman**ı değil, **kimin kimi çağırdığını** gösterir.

3.9 assert ile birim testi

Birim testi (unit test), tek bir fonksiyonu bilinen girdilerle çağırıp sonucu beklenenle karşılaştırmaktır. `<assert.h>`'deki `assert(koşul)` doğruysa hiçbir şey yapmaz; yanlışsa stderr'e dosya, satır, fonksiyon ve koşulu yazar ve `abort()` ile durur. `H08_K15_assert_yakalar.c` bilinçli olarak hatalı bir `is_leap_year` (yalnızca `return yil % 4 == 0;`) içerir; `assert(!is_leap_year(1900));` bunu yakalar:

ASSERT HATAYI YAKALAR; -DNDEBUG İLE KAPATILINCA HATA GÖRÜNMEZ — GERÇEK

```
$ ./assert_yakalar; echo "[çıkış kodu: $?]"
1. test geçti: 2024 artık yıl
assert_yakalar: H08_K15_assert_yakalar.c:19: main: Assertion `!is_leap_year(1900)'
failed.
bash: line 1:  9232 Aborted                 ./assert_yakalar
[çıkış kodu: 134]
$ ./assert_yakalar_ndebug     # -DNDEBUG ile derlenmiş
Tüm testler geçti.
[çıkış kodu: 0]
```

`abort()` programı **SIGABRT** (6) ile bitirir; kabuk 128 + 6 = **134** bildirir. İki uyarı: (1) `-DNDEBUG` ile derlenince `assert`'ler önişlemcide **silinir**: aynı hatalı program "Tüm testler geçti." yazdı; bu yüzden `assert` içine yan etkili ifade (`assert(scanf(...) == 1)`) yazılmaz. (2) `assert` programcı hataları içindir, **kullanıcı girdisini** doğrulamak için değil: yanlış not giren kullanıcıya program çökmeden yeniden sormalıdır (H6).

Testleri tasarlamak: normal, sınır, hatalı girdi ve kayan nokta

Her fonksiyon için **normal** (is_prime(97)), **sınır** (0, 1, 2, `INT_MAX`) ve **hatalı/beklenmedik** (−7) girdiler denenir (H5'teki sınır değer çözümlemesi). Kayan noktada `==` kullanılmaz: `H08_K16_kayan_tolerans.c` "0.1 + 0.2 = 0.30000000000000004", "a == 0.3 : 0", "yakin(a, 0.3) : 1" yazar (gerçek çıktı).

Testlerde `yakin(x, y)` ($|x - y| < 10^{-9}$) sorulur; büyük değerlerde göreli tolerans gerekir. Örnek 12 (24 doğrulama) ve Mini-OBS v0.8 (37 doğrulama) bu yapıdadır.

3.10 Kaputun altında: bir çağrıda neler olur?

> KAPUTUN ALTINDA · **call, ret ve yığın çerçevesi: giriş düzeyi (ayrıntısı H9'da)**
>
> Çağrı işlemci düzeyinde üç iştir: (1) argümanlar çağrılanın bulacağı yere konur; (2) `call`, **dönüş adresini** (sonraki komutun adresini) yığına (stack) iter ve fonksiyona atlar; (3) `ret` bu adresi yığından alıp geri döner. Her çağrı yığında bir **yığın çerçevesi** (stack frame) kurar: parametre kopyaları ve yerel değişkenler buradadır ve dönüşte kalkar (Örnek 10'da `takas`'ın değişkenleri bu yüzden kaybolur).

Argümanların yeri **çağrı kuralına** (calling convention, ABI) bağlıdır. x86-64 Linux'un **System V AMD64 ABI**'sinde ilk altı tamsayı argüman **yazmaçlarla** gider (`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`; `int` için `edi`, `esi` ...), kayan noktalılar `xmm0`–`xmm7` ile; dönüş değeri `rax`/`eax` ya da `xmm0`'dadır (Bryant & O'Hallaron, 2016). "Argümanlar kopyalanır" doğrudur; "yığına itilir" genellemesi bu platformda **yanlıştır**.

`H08_K07_asm_cagri.c`'deki `topla(3, 4)` çağrısı için `main` yalnızca üç komut üretir: `mov esi, 4`, `mov edi, 3`, `call topla`. Çağrılan `topla`'nın çevirisi (Intel sözdizimi):

TOPLA'NIN GERÇEK GCC -O0 -S ÇIKTISI (TAM METİN KODLAR/CİKTİLAR/H08_K07_ASM_CAGRİ_O0_ASM.TXT)

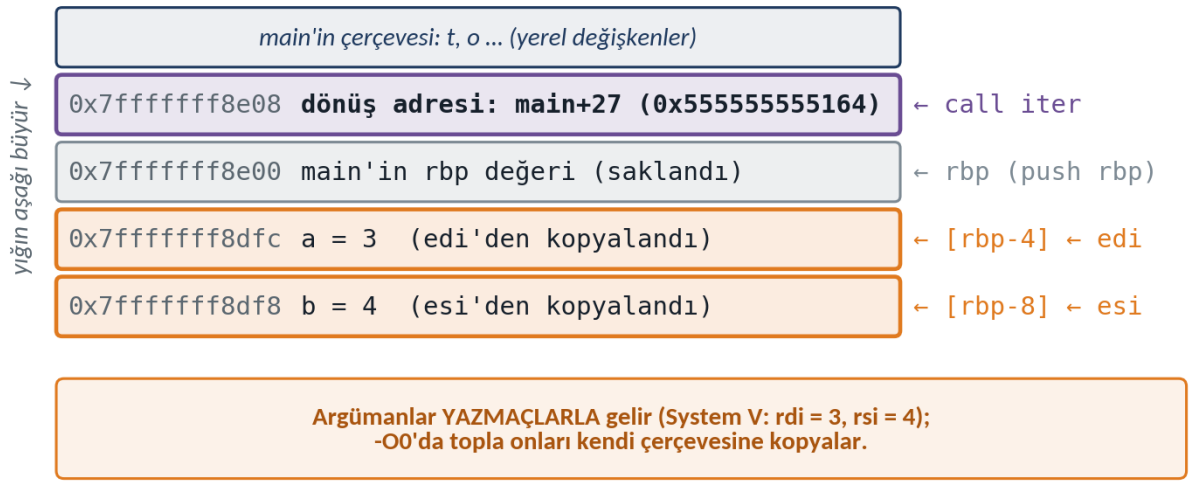
```
$ gcc -std=c17 -O0 -S -masm=intel -fno-asynchronous-unwind-tables -fno-pie
H08_K07_asm_cagri.c -o -
topla:
    endbr64
    push  rbp
    mov  rbp, rsp
    mov  DWORD PTR [rbp-4], edi
    mov  DWORD PTR [rbp-8], esi
    mov  edx, DWORD PTR [rbp-4]
    mov  eax, DWORD PTR [rbp-8]
    add  eax, edx
    pop  rbp
    ret
```

Yani 3 `edi`'ye, 4 `esi`'ye konur (`double` argümanlar ise `xmm0`/`xmm1` ile gider). `topla` `push rbp` ve `mov rbp, rsp` ile çerçevesini kurar, sonra `mov DWORD PTR [rbp-4], edi` ve `[rbp-8], esi` ile argümanları **kendi çerçevesine kopyalar**: parametrenin "ayrı kutu" olmasının makinedeki karşılığı. Toplam `eax`'te döner; `pop rbp` ve `ret`.

gdb (GNU hata ayıklayıcısı) ile `topla`'ya girildiği anda durdurduk (oturumun tamamı `kodlar/ciktilar/H08_K07_asm_cagri_gdb.txt`; Şekil 8.6).

`rdi` = 3, `rsi` = 4: argümanlar yazmaçlarda. `rbp + 8`'de (0x7fffffff8e08) **dönüş adresi** 0x555555555164 = `main+27` durur; `objdump` bunun `call topla`'dan hemen sonraki komut olduğunu doğrular (`kodlar/ciktilar/`). `a`'nın adresi 0x7fffffff8dfc = rbp − 4: `topla`'nın çerçevesi.

Şekil 8.6
topla(3, 4) çağrısında yığın: dönüş adresi, saklanan rbp ve parametre kopyaları (gdb verileriyle)



Windows'ta farklı. Microsoft x64 çağrı kuralında ilk dört tamsayı argüman `rcx`, `rdx`, `r8`, `r9` ile gider ve çağıran 32 baytlık **gölge alan** (shadow space) ayırır (Microsoft, t.y.). Aynı kaynağın `gcc -mabi=ms -S` çevirisinde (yalnızca gösterim) `main` `mov ecx, 3`, `mov edx, 4` ve `sub rsp, 48` yazıyor; C kaynağı aynıdır, farkı derleyici halleder.

`-O2` ile `topla` `lea eax, [rdi+rsi]` ve `ret`'e iner ve `main` onu hiç çağırmaz: derleyici fonksiyonu **satır içine alıp** (inlining) sonucu önceden hesaplar (`mov edx, 7`). Çerçeveleri **Python Tutor**'un C görselleştiricisiyle (https://pythontutor.com/c.html) adım adım izleyebilirsiniz (Python Tutor'un ilk sürümü: Guo, 2013; C/C++ desteği: Guo, 2021): `H08_02_kare.c` ve `H08_10_takas_denemesi.c` ile deneyin.

4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır (`kodlar/H08_01_…` – `H08_13_…`); hepsi `gcc -std=c17 -Wall -Wextra -pedantic -g` ile **sıfır uyarıyla** derlenir, AddressSanitizer ve UBSan ile temiz çalışır; çıktılar gerçektir. Örnek 6 ve 13 standart çözümlü örnek şablonuyla işlenmiştir.

4.1 ★ basamaklar: ilk fonksiyonlar (Örnek 1–4)

Örnek 1 ★ (`selamla()`) ve **Örnek 2 ★** (`kare(x)`) Bölüm 3.2'de incelendi. **Değiştir:** `void selamla(int hafta)` yazın; `kare`'nin prototipini silip derleyicinin iletisini okuyun.

Örnek 3 ★ — max2 ve max3: bileşim

H08_03_max2_max3.c · satır 32–36

```c
/* Bileşim: önce a ile b'nin büyüğü, sonra onunla c'nin büyüğü. */
int max3(int a, int b, int c)
{
    return max2(max2(a, b), c);
}
```

`max2` ve `max3`'teki `a`, `b` parametreleri **ayrı** değişkenlerdir (her fonksiyonun parametreleri kendine aittir; kapsam, H9). `max3(4, 9, 2)`'de önce `max2(4, 9)` = 9, sonra `max2(9, 2)` = 9; gerçek çıktı: "max3(4, 9, 2) = 9", "max3(7, 7, 7) = 7", "max3(-1, -5, 0) = 0". **Değiştir:** `min3`'ü ve `max4`'ü iki farklı bileşimle yazın.

Örnek 4 ★ — is_even ve is_leap_year: bool döndüren doğrulama

H08_04_cift_artik_yil.c · satır 43–50

```c
void yil_raporu(int yil)
{
    if (is_leap_year(yil)) {
        printf("%d: artık yıl (366 gün)\n", yil);
    } else {
        printf("%d: artık yıl değil (365 gün)\n", yil);
    }
}
```

`H05_05_artik_yil.c`'deki ifade **bir kez** yazılıp adlandırıldı. `yil_raporu` yazdırır, `is_leap_year` karar verir: hesap ile çıktının ayrılması test etmeyi kolaylaştırır (Örnek 12, Alıştırma 8.9). Test yılları kuralın üç dalını kapsar: "1900: artık yıl değil (365 gün)", "2000: artık yıl (366 gün)", "2024: artık yıl (366

gün)", "2026: artık yıl değil (365 gün)", "2100: artık yıl değil (365 gün)"; `is_even(10)`, `is_even(7)`, `is_even(-4)` için 1, 0, 1 (gerçek çıktı).

4.2 Örnek 5 ★★ — H6 kodunu fonksiyona taşımak: faktoriyel(n) ve us_al(taban, us)

H08_05_faktoriyel_us.c · satır 27–36

```c
/* Ön koşul : 0 <= n <= 20 (21! 64 bite sığmaz; bkz. H06_05).
 * Son koşul: n! değerini döndürür (0! = 1). */
unsigned long long faktoriyel(int n)
{
    unsigned long long carpim = 1;      /* çarpım biriktirici (H6) */
    for (int i = 2; i <= n; i++) {
        carpim *= i;
    }
    return carpim;
}
```

H6'daki döngüler (`H06_05_faktoriyel_tasma.c`, `H06_06_us_alma.c`) neredeyse hiç değişmeden gövdelere taşındı (`us_al` aynı kalıpta: `long long sonuc = 1;`, `sonuc *= taban;`); değişen **çevreleridir**: girdi parametreden gelir, sonuç `return` ile döner. Gerçek çıktı: "20! = 2432902008176640000", "2^10 = 1024", "C(5, 2) = 5! / (2! * 3!) = 10": aynı fonksiyon bir ifadede üç kez kullanıldı (20 sınırı H6'nın taşma deneyinden). **Değiştir:** `kombinasyon(n, k)` yazın; n = 21'de neden yanlış?

4.3 Amiral örnek — Örnek 6 ★★: is_prime(n), H7 kodunun yeniden düzenlenmesi

1. Problem. H7'de asallık testi `main`'in içindeydi (`H07_06_asal_naif.c`, `H07_11_asal_karekok_sure.c`); başka bir programda kullanmak için kopyalamak gerekiyordu. Hızlı sürümü her yerden çağrılabilen, test edilebilen bir `bool is_prime(int n)`'e dönüştürün; davranış değişmemeli: bu bir **yeniden düzenlemedir** (refactoring).

2. Girdi, çıktı ve kısıtlar. Girdi herhangi bir `int`; çıktı `bool` (asalsa `true`), ekrana hiçbir şey yazmaz. Ön koşul yoktur: **her** `int` için yanıt verilmeli; 0, 1 ve negatifler asal değil, 2 tek çift asaldır; 2 147 483 647 ($2^{31}-1$) asaldır ve döngü koşulu orada taşmamalıdır. **3. Algoritma:** `n < 2 → yanlış; n = 2 → doğru; n çift → yanlış; d ← 3; d ≤ n / d iken: n mod d = 0 → yanlış; d ← d + 2; sonunda doğru.`

4. C kodu.

H08_06_is_prime.c · satır 19–24

```c
    for (int n = 1; n <= 50; n++) {
        if (is_prime(n)) {              /* koşul bir cümle gibi okunur */
            printf(" %d", n);
            adet++;
        }
    }
```

H08_06_is_prime.c · satır 33–54

```c
/* Ön koşul : yok (her int için tanımlı).
 * Son koşul: n asalsa true, değilse false döndürür.
 * Yöntem   : 2 dışındaki çiftleri ele; 3'ten başlayıp yalnızca tek
 *            bölenleri dene; d <= n / d (d * d <= n yerine: taşma yok). */
bool is_prime(int n)
{
    if (n < 2) {
        return false;                   /* 0, 1 ve negatifler asal değil */
    }
    if (n == 2) {
        return true;                    /* tek çift asal */
    }
    if (n % 2 == 0) {
        return false;                   /* diğer bütün çiftler */
    }
    for (int d = 3; d <= n / d; d += 2) {
        if (n % d == 0) {
            return false;               /* bölen bulundu: hemen dön */
        }
    }
    return true;                        /* hiçbir tek bölen yok */
}
```

5. Satır satır. Prototip sayesinde `main` fonksiyonu nasıl çalıştığını bilmeden kullanır; satır 20'in koşulu "eğer n asalsa" diye okunur. Fonksiyonun birden çok `return`'ü vardır: sınır durumları erken elenir. Satır 48: `d * d <= n` yerine `d <= n / d`; anlamı aynı ama ilki n = 2 147 483 647'de d = 46 341 olunca taşar (tanımsız davranış). UBSan ile derlenen `H08_K18_karekok_tasma.c`: "runtime error: signed integer overflow: 46341 * 46341 cannot be represented in type 'int'". Bölen bulununca `return false` döngüyü ve fonksiyonu birlikte bitirir; H7'deki `asal = 0; break;` bayrağına gerek kalmaz.

6. İzleme tablosu. `is_prime(91)` ve `is_prime(97)` için döngü denetimleri (Tablo 8.2):

Tablo 8.2
is_prime izleme tablosu (n / d tamsayı bölmesidir)

Çağrı	d	koşul d ≤ n / d	n % d	sonuç
is_prime(91)	3	3 ≤ 30 → doğru	1	d += 2
is_prime(91)	5	5 ≤ 18 → doğru	1	d += 2
is_prime(91)	7	7 ≤ 13 → doğru	0	**return false**
is_prime(97)	3	3 ≤ 32 → doğru	1	d += 2
is_prime(97)	5	5 ≤ 19 → doğru	2	d += 2
is_prime(97)	7	7 ≤ 13 → doğru	6	d += 2
is_prime(97)	9	9 ≤ 10 → doğru	7	d += 2
is_prime(97)	11	11 ≤ 8 → yanlış	—	**return true**

7–8. Örnek çalıştırma ve test tablosu (gerçek çıktılar; aynı testler Örnek 12'de `assert` ile otomatikleşir):

Tablo 8.3
Örnek 6 test tablosu

Test	Girdi	Beklenen	Gerçek	Sonuç
Normal (asal)	97	1	1	✓
Normal (asal görünümlü)	91 = 7 · 13	0	0	✓
Sınır: liste	1..50	15 asal, 1 yok	15 asal; ilk asal 2	✓
Sınır: INT_MAX	2147483647	1, taşma yok	1 (UBSan temiz)	✓
Hatalı girdi	-7	0	0	✓

9. Değiştir-genişlet. (a) `asal_say(alt, ust)`'u `is_prime` ile yazın (1–1000 arası kaç asal?). (b) `sonraki_asal(n)`. (c) Gövdeyi H7'nin naif sürümüyle değiştirin: `main` ve testler değişmeden çalışmalı (kara kutu).

4.4 Fonksiyonlar birlikte çalışır (Örnek 7–9)

Örnek 7 ★★ — ebob ve ekok: bir fonksiyon diğerini kullanır

H08_07_ebob_ekok.c · satır 39–45

```
39  /* Ön koşul : a > 0, b > 0.
40   * Son koşul: a ile b'nin en küçük ortak katı.
41   * ebob'u KULLANIR; önce böl, sonra çarp: ara sonuç küçük kalır. */
42  long long ekok(int a, int b)
43  {
44      return (long long)a / ebob(a, b) * b;
45  }
```

Gerçek çıktı (girdi 1071 462): "EBOB(1071, 462) = 21", "EKOK(1071, 462) = 23562". H6'daki Öklid döngüsü (`H06_11_ebob_ekok.c`) `ebob`'un gövdesi oldu: `while (b != 0) { int r = a % b; a = b; b = r; } return a;`. `ebob` parametrelerini değiştirir ama `main`'deki `a`, `b` aynı kalır (değerle aktarım). `ekok` işi `ebob`'a devreder; önce böldüğü için ara sonuç küçük kalır.

Örnek 8 ★★ — Birim dönüştürücü: menü + fonksiyonlar

`H08_08_birim_donusturucu.c`'de H7'nin menü iskeleti (`do-while` + `switch`) aynen durur; her `case` iki çağrıya indi: `x = deger_oku();` ve sonucu `c_to_f(x)` gibi bir dönüşümle yazan `printf`. Dönüşümler (`c_to_f`, `km_to_mil` …) yalnızca hesap yapar; `deger_oku` geçerli sayı gelene kadar sorar ve EOF'ta `exit(EXIT_FAILURE)` ile bitirir (çıkış kodu 1).

Örnek 9 ★★ — basamak_toplami, ters_cevir, is_palindrome

`H08_09_basamak_palindrom.c`'de H6'nın iki döngüsü `basamak_toplami(n)` ve `ters_cevir(n)` oldu; `is_palindrome(n)` ise tek satırdır: `return ters_cevir(n) == n;`. Kendi döngüsünü yazmaz, "tersi kendisine eşit mi?" sorusunu `ters_cevir`'e sorar: küçük fonksiyonlardan büyük çözüm. Gerçek çıktılar: 12321 → "evet", 2026 → "hayır", 1230'un tersi 321 (sondaki 0 kaybolur; sözleşmeye yazılmalı), 1999999999'un tersi 9999999991: `int`'e sığmadığı için dönüş tipi `long long`.

4.5 Değerle aktarımın sınırı ve dizisiz sayma (Örnek 10–11)

Örnek 10 ★★ — "Neden çalışmıyor?" takas(a, b) denemesi

H08_10_takas_denemesi.c · satır 15–32

```c
int main(void)
{
    int a = 3, b = 7;
    printf("main  (önce) : a = %d, b = %d   &a = %p\n", a, b, (void *)&a);
    takas(a, b);
    printf("main  (sonra): a = %d, b = %d   <- değişmedi!\n", a, b);
    return 0;
}

/* AMAÇ: a ile b'yi değiş tokuş etmek. Neden işe yaramıyor? */
void takas(int a, int b)
{
    printf("takas (giriş): a = %d, b = %d   &a = %p\n", a, b, (void *)&a);
    int gecici = a;                    /* H3'teki geçici değişkenli takas */
    a = b;
    b = gecici;
    printf("takas (çıkış): a = %d, b = %d\n", a, b);
}
```

H3'teki geçici değişkenli takas bir fonksiyona kondu: **içeride** takas başarılı ("takas (çıkış): a = 7, b = 3"), ama `main`'e dönünce değerler **eskisi gibi** ("main (sonra): a = 3, b = 7"; tam çıktı `kodlar/ciktilar/H08_10_takas_denemesi.txt`). `takas(a, b)` değişkenleri değil **değerlerinin kopyalarını** gönderdi; iki `a` ayrı değişkenlerdir: `main`'inki 0x7ffd05ff2fc0, `takas`'ınki 0x7ffd05ff2f9c adresinde (Şekil 8.7). Takas kopyalarda yapıldı ve çerçeveyle birlikte silindi. Adresler her çalıştırmada değişir (ASLR, H3; ikinci çalıştırmada 0x7ffe5f1d5950).

Şekil 8.7
takas(a, b) neden çalışmıyor: kopyalar takas edilir, main'in değişkenleri değişmez (gerçek adreslerle)



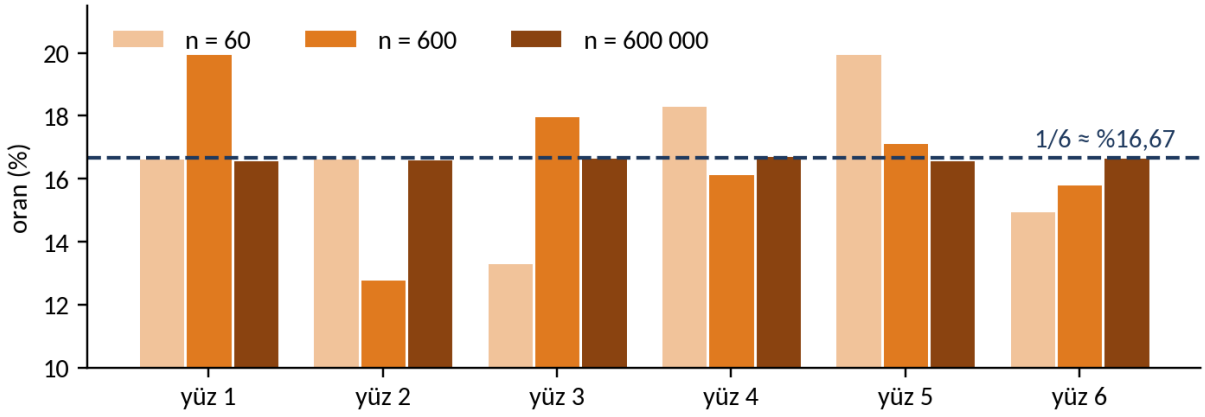
> **AP-2 KÖPRÜSÜ · Bilinçli açık soru: takas nasıl çalışır hâle gelir?**
>
> Çağıranın değişkenini değiştirmek için değeri değil **adresi** göndermek gerekir: `takas(&a, &b)`. Bunun aracı **işaretçidir**; H12'de `void takas(int *a, int *b)` olarak çözülecek. Sıralama algoritmalarının temel işlemi takastır (AP-2). O zamana kadar değişmiş değer almanın yolu `return`'dür: Örnek 13'te bakiye böyle güncellenir.

Örnek 11 ★★ — Zar simülasyonu ve altı sayaçla frekans

`H08_11_zar_frekans.c` n kez `switch (zar_at())` yapar ve `case 1: s1++; break;` kalıbıyla altı sayaçtan birini artırır. `zar_at() return rand() % 6 + 1;` ile 1–6 döndürür ve `switch`'in koşulu olur. Dizi henüz olmadığından **altı ayrı sayaç** (`s1` … `s6`) gerekti; H10'daki **frekans dizisi** (`int sayac[7]`) bunu tek satıra indirecek. Kayıtlı çıktılar `-DSABIT_TOHUM=2026` ile alındı; öğrenci sürümü `srand((unsigned)time(NULL))` kullanır.

Büyük sayılar yasası, gerçek veriyle. 60 atışta oranlar %13,33 ile %20,00 arasında dağıldı; 600 atışta %12,83–%20,00; 600 000 atışta ise hepsi %16,60–%16,76 aralığına, yani 1/6 ≈ %16,67'nin çok yakınına toplandı (Şekil 8.8).

Şekil 8.8
Atış sayısı arttıkça her yüzün oranı 1/6'ya yaklaşır (Örnek 11'in gerçek çıktıları)



4.6 Örnek 12 ★★★ — assert ile is_prime ve ebob testleri

H08_12_assert_testleri.c · satır 36–50

```
36  int test_is_prime(void)
37  {
38      assert(is_prime(7));                /* normal */
39      assert(is_prime(97));
40      assert(!is_prime(91));              /* 7 * 13: asal "görünümlü" */
41      assert(!is_prime(0));               /* sınır */
42      assert(!is_prime(1));
43      assert(is_prime(2));
44      assert(is_prime(3));
45      assert(!is_prime(4));
46      assert(is_prime(2147483647));       /* INT_MAX: taşma olmamalı */
47      assert(!is_prime(-7));              /* beklenmedik: negatif */
48      printf("is_prime   : 10 test geçti\n");
49      return 10;
50  }
```

Gerçek çıktı: “is_prime: 10 test geçti”, “ebob/ekok: 8 test geçti”, “ortalama: 6 test geçti”, “Tüm testler geçti (24 doğrulama).”. Her **test fonksiyonu** tek bir fonksiyonu sınar ve doğrulama sayısını döndürür; testler sözleşmeden türetildi: normal (97), asal görünümlü bileşik (91), sınırlar (0–4, `INT_MAX`), beklenmedik girdi (−7). `test_ebob_ekok` `int`’e sığmayan bir EKOK’u (4 294 967 294) dener. Fonksiyonlar Örnek 6–7’den kopyalandı; H9’da, Mini-OBS v0.9’un `test_obs.c` dosyasında olduğu gibi, testler ayrı bir dosyaya taşınıp fonksiyonların bulunduğu dosyayla birlikte bağlanabilir. `fabs` için `-lm` gerekir.

4.7 Amiral örnek — Örnek 13 ★★★: ATM programının fonksiyonlarla yeniden yapılandırılması

1. Problem. H7’nin ara projesi `H07_14_atm_menu.c`’yi, davranışını ve çıktısını **hiç değiştirmeden**, yukarıdan aşağıya tasarımla fonksiyonlara bölün. Kurallar aynı: PIN 1234, en çok 3 deneme (bloke → çıkış kodu 2); bakiye 1000 TL; yatırma 1–10 000 TL; çekme 10 TL’nin katı ve bakiyeyi aşmayan (0 = iptal); EOF’ta “Girdi sona erdi.” ve çıkış kodu 1.

Tablo 8.4
Örnek 13’ün girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• PIN denemeleri • menü seçimleri (1–4) • tutarlar (geçersiz girdiler dahil)	• pin_dogrula: en çok 3 deneme • menü döngüsü: seçime göre alt fonksiyon • bakiye fonksiyonlara değerle gider, yenisi return ile döner • tek girdi okuyucu: tamsayi_oku	• H7 ile birebir aynı ekran çıktısı • aynı çıkış kodları (0, 1, 2)

2. Çözümleme. H7 sürümünde `scanf` + EOF denetimi + tampon temizleme bloğu **dört kez** yazılmıştı (PIN, menü, yatırma, çekme); tek fark, sayı olmayan girdide verilen “geçersiz” değerdi (−1, 0, 0, −1). Fark bir **parametreye** dönüşür: `int tamsayi_oku(int gecersiz)`. **3. Tasarım:** Bölüm 3.8’deki iyileştirme ve yapı şeması (Şekil 8.5). Bakiye `main`’in yerel değişkenidir; değiştiren fonksiyonlar onu **değerle alır**, yenisini **döndürür**: `bakiye = para_yatir(bakiye);` (işaretçi gerekmez, H12).

4. C kodu (kesitler; tamamı `kodlar/H08_13_atm_fonksiyonlu.c`):

H08_13_atm_fonksiyonlu.c · satır 40–67

```
40      printf("===== AP-1 BANKASI ATM =====\n");
41      if (!pin_dogrula()) {
42          printf("Kartınız bloke edildi. Lütfen şubenize başvurun.\n");
43          return 2;
44      }
45
46      int secim;
47      do {
48          menu_goster();
49          secim = tamsayi_oku(0);         /* sayı değilse 0: geçersiz seçim */
50          switch (secim) {
51          case 1:
52              bakiye_goster(bakiye);
53              break;
54          case 2:
55              bakiye = para_yatir(bakiye);    /* kopya gider, yenisi döner */
```

```
56              break;
57          case 3:
58              bakiye = para_cek(bakiye);
59              break;
60          case 4:
61              printf("İyi günler dileriz.\n");
62              break;
63          default:
64              printf("  Geçersiz seçim: 1-4 arası bir sayı girin.\n");
65              break;
66          }
67      } while (secim != 4);
```

H08_13_atm_fonksiyonlu.c · satır 84–97

```
84  int tamsayi_oku(int gecersiz)
85  {
86      int deger;
87      int okunan = scanf("%d", &deger);
88      if (okunan == EOF) {
89          printf("\nGirdi sona erdi.\n");
90          exit(EXIT_FAILURE);             /* EXIT_FAILURE = 1 */
91      }
92      if (okunan == 0) {
93          satir_temizle();
94          return gecersiz;
95      }
96      return deger;
97  }
```

5. Satır satır. `main` yalnızca **ne** yapılacağını söyler. Satır 41: `bool` döndüren `pin_dogrula` koşul olarak okunur. Satır 55: `para_yatir` (1–10 000 arası tutar isteyen bir `do-while`) bakiyenin **kopyasını** alır, artırıp döndürür; `main` onu atar. Atama unutulsaydı yatırılan para kaybolurdu (HA3). `tamsayi_oku` EOF'ta `exit(EXIT_FAILURE)` ile H7'deki `return 1;` ile aynı sonucu verir, ama artık **tek yerde**. Kurallar dosya kapsamındaki sabitlerdir (`DOGRU_PIN` …; kapsam, H9).

6. İzleme: bakiye çağrılar arasında nasıl dolaşır? Başlıktaki örnek girdi (`1111 1234 1 3 250 2 100 4`) için (Tablo 8.5):

Tablo 8.5
Örnek 13: bakiye değerle gider, return ile döner (girdi: 1111 1234 1 3 250 2 100 4)

Adım	Çağrı	Parametre (kopya)	Dönüş değeri	main'deki bakiye
1	pin_dogrula()	—	true	1000
2	bakiye_goster(bakiye)	bakiye = 1000	(void)	1000
3	para_cek(bakiye) çağrılır	bakiye = 1000	(henüz dönmedi)	1000
3a	cekilecek_tutar_oku(bakiye)	bakiye = 1000	250	1000
3	para_cek döner	—	750	**750**
4	para_yatir(bakiye)	bakiye = 750	850	**850**
5	tamsayi_oku(0) → 4	gecersiz = 0	4 (çıkış)	850

7–8. Örnek çalıştırma ve test. Başlıktaki girdinin gerçek çıktısı `kodlar/ciktilar/H08_13_atm_fonksiyonlu.txt`'dedir (son satır "İyi günler dileriz."). Davranış gerçekten aynı mı? Dört senaryo hem `H07_14_atm_menu.c` hem bu programla çalıştırıldı; çıktılar `diff` ile karşılaştırıldı, çıkış kodları kaydedildi (Tablo 8.6).

Tablo 8.6
Örnek 13: H7 sürümüyle karşılaştırma (gerçek diff çıktıları, kodlar/ciktilar/H08_13_atm_fonksiyonlu.txt)

Senaryo	Girdi	Çıkış kodu (H7 / H8)	diff sonucu
başlıktaki örnek: bir hatalı PIN, bakiye, çekme, yatırma	1111 1234 1 3 250 2 100 4	0 / 0	fark yok
geçersiz girdiler	abc 1234 x 9 2 0 abc 20000 500 3 15 -10 abc 5000 1490 3 0 1 4	0 / 0	fark yok
üç hatalı PIN: bloke	1 2 abc	2 / 2	fark yok
menüde girdi biter (EOF)	1234 1	1 / 1	fark yok

9. Değiştir–genişlet. (a) "Son işlemler" özelliği ekleyin: hangi fonksiyonlar değişir? (b) Günlük çekme sınırı (2000 TL): toplam nerede tutulmalı, hangi fonksiyona parametre olmalı? (c) Yapı şemasını güncelleyin. Değişiklik ilgili fonksiyonun içinde kalır: yeniden yapılandırmanın değeri budur.

5. Hata Avcısı

Bazı fonksiyon hatalarını derleyici **durdurur**, bazılarını yalnızca **uyarır**, bazılarını hiç fark etmez. Tablo 8.7 özetler; iletiler GCC 13.3 (`-std=c17 -Wall -Wextra -pedantic`) ve Clang 18'in gerçek çıktılarıdır (`kodlar/hata_avcisi/ciktilar/`).

Tablo 8.7
Haftanın Hata Avcısı özeti (iletiler gerçektir)

#	Hata	Derleyici / belirti	Neden	Çözüm
HA1	Prototip eksik	GCC 13: uyarı (double dönüşte hata); GCC 14+, Clang 16+: hata	int dönüş varsayılır	Prototipi çağrıdan önce yaz
HA2	return unutulmuş	uyarı: control reaches end of non-void function	Tanımsız davranış	Her yol return'e ulaşsın
HA3	Dönüş değeri kullanılmamış	kendi fonksiyonumuzda uyarı yok	Sonuç atanmaz, kaybolur	Sonucu ata: x = f(x);
HA4	Parametre sayısı / tipi	sayı: hata; tip: sessiz dönüşüm	Tip örtük dönüşür	İmzaya uy; -Wconversion
HA5	Tanım başlığına ;	hata: expected identifier or '(' before '{'	Blok sahipsiz kalır	Tanımda ; yok
HA6	Fonksiyon içinde fonksiyon	GCC: uzantı (-pedantic uyarır); Clang: hata	ISO C'de iç içe tanım yok	Yan yana yaz

HA1 · Prototip eksikliği ve örtük bildirim

GCC 13.3 UYARI VERİP DERLER; CLANG 18 DERLEMEZ — GERÇEK

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H08_HA1_prototip_eksik.c -o ha1
H08_HA1_prototip_eksik.c: In function ‘main’:
H08_HA1_prototip_eksik.c:13:29: warning: implicit declaration of function ‘kup’ [-
Wimplicit-function-declaration]
$ ./ha1
kup(3) = 27
$ clang -std=c17 … H08_HA1_prototip_eksik.c
13:29: error: call to undeclared function 'kup'; ISO C99 and later do not support
implicit function declarations [-Wimplicit-function-declaration]
```

`H08_HA1_prototip_eksik.c`'de `main`, kendisinden sonra tanımlanan `kup`'u prototipsiz çağırır. GCC uyarıya rağmen derledi, sonuç **şans eseri** doğru: derleyici C89'un örtük kuralıyla "`int` döndüren fonksiyon" varsaydı ve `kup` gerçekten `int` döndürüyordu. Dönüş `double` olsaydı (`H08_HA1_prototip_eksik_double.c`): "error: conflicting types for 'ortalama'; have 'double(int, int)'"; GCC ayrıca `%.2f`'e `int` gittiğini uyarır: varsayılan `int` yanlış yorumun kendisidir. GCC 14 ve Clang 16'dan bu yana örtük bildirim C99 ve sonrası için varsayılan olarak **hatadır** (GCC Team, 2024; LLVM Project, 2023). **Kural:** her fonksiyon için prototip (`…_duzeltilmis.c`).

HA2 · return unutulması: tanımsız davranış

`hata_avcisi/H08_HA2_return_unutma.c · satır 19–24`

```
19  int max2(int a, int b)
20  {
21      if (a > b) {
22          return a;
23      }
24  }
```

`a <= b` olduğunda `max2` hiçbir `return`'e ulaşmaz; değeri kullanmak **tanımsız davranıştır** (ISO/IEC, 2018). `max2(4, 9)` GCC `-O0` ile 4, `-O2` ile 0, Clang ile 32525 verdi: hiçbiri 9 değil, hiçbirine güvenilemez. GCC'nin uyarısı ("warning: control reaches end of non-void function [-Wreturn-type]", `-Wall` içinde) tam bu yolu gösterir. Düzeltme: **her** yol bir `return`'e ulaşmalı.

HA3 · Dönüş değerinin kullanılmaması

`H08_HA3_donus_kullanilmiyor.c`'de `toupper(harf);` ve `us_al(2, 10);` satırları sonuçlarını hiçbir yere atamaz. Gerçek çıktı: "Büyük harf: k", "2^10 = 1". `toupper` ve `us_al` argümanı **değiştirmez**, yeni değer **döndürür**; atanmayan değer kaybolur. glibc'de `toupper` makro olduğundan GCC "statement with no effect" dedi; ama `us_al(2, 10);` için **uyarı yok**. C23'ün `[[nodiscard]]` özniteliği bunu çözer (`H08_HA3_nodiscard_gosterim.c`, `-std=c2x`): "warning: ignoring return value of 'us_al', declared with attribute 'nodiscard' [-Wunused-result]". Düzeltme: `harf = toupper(harf);`, `sonuc = us_al(2, 10);`.

HA4 · Parametre sayısı ve tipi uyuşmazlığı

Prototip varken argüman sayısı yanlışsa derleyici durur: `max2(5)` → "error: too few arguments to function 'max2'". Tip uyuşmazlığı ise sessizdir: `int kare(int x)` iken `kare(2.5)` argümanı `int`'e çevirir ve "kare(2.5) = 4" yazar; yalnızca `-Wconversion` uyarır ("warning: conversion from 'double' to 'int' may change value [-Wfloat-conversion]"). Düzeltme: doğru tip (`double kare_d(double x)`). `()` ile bildirilen fonksiyonda ikisi de denetlenmez (Bölüm 3.4).

HA5 · Tanım başlığının sonuna noktalı virgül ve HA6 · iç içe fonksiyon

HA5'te (`H08_HA5_noktali_virgul.c`) `int kare(int x);` satırının altında `{ return x * x; }` durur: noktalı virgül başlığı **prototipe** çevirir, blok sahipsiz kalır: GCC "error: expected identifier or '(' before '{' token" der. İletideki satır (11) sahipsiz `{`'nin satırıdır; neden çoğu zaman bir **önceki** satırdadır (H1). HA6'da (`H08_HA6_ic_ice_fonksiyon.c`) `int kare(int x) { return x * x; }` tanımı `main`'in **içine** yazılmıştır.

GCC: "warning: ISO C forbids nested functions [-Wpedantic]"; program derlenip "kare(5) = 25" yazdı. Clang 18: "error: function definition is not allowed here". ISO C'de fonksiyon içinde fonksiyon tanımlanamaz; GCC bunu bir **uzantı** (nested functions) olarak kabul eder (`-pedantic` uyarır, `-pedantic-errors` durdurur). GCC'de çalışması taşınabilir olduğunu göstermez. Düzeltme: fonksiyonlar dosya düzeyinde, yan yana.

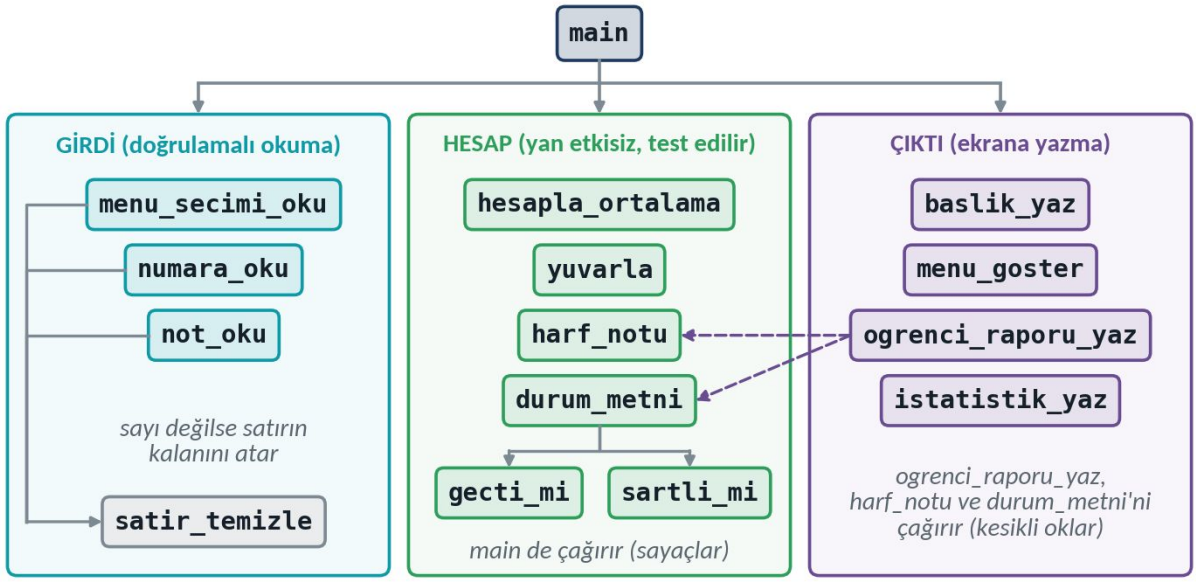
6. Mühendis Gözüyle: Modülerlik, Sözleşme ve Test

- **Bir fonksiyon, bir iş.** Ne yaptığını "ve" kullanmadan tek cümleyle söyleyebilmelisiniz; hesap yapan fonksiyon yazmaz, okumaz: **yan etkisiz** fonksiyon kolay test edilir.
- **İmza ve sözleşme belgedir.** `int para_yatir(int bakiye)` girdisini ve çıktısını söyler; global değişkenler bunu saklar (H9). Her ön koşul sınırı bir sınır testidir.
- **Tanımsız davranışa sıfır tolerans.** Taşabilecek ifadeyi yeniden yazın (`d <= n / d`), her yolu `return`'e ulaştırın (HA2), uyarıları hata gibi okuyun (`-Werror`).
- **Yeniden düzenlemede davranış korunur.** Örnek 13 ve Mini-OBS v0.8'de iki sürümü `diff` ile karşılaştırdık; endüstride bunu **regresyon testleri** yapar.

7. Mini-OBS v0.8: Fonksiyonlara Ayrıştırma ve assert Testleri

v0.7 menü döngüsünü kurmuştu ama her şey tek `main`'deydi ve not doğrulama bloğu üç kez yazılmıştı. **v0.8, v0.7'nin davranışını aynen koruyarak** programı fonksiyonlara böler (Şekil 8.9). Kaynak ve tasarım belgesi: `91_Proje_MiniOBS/v0.8/` (kopyası `kodlar/proje/`).

Şekil 8.9
Mini-OBS v0.8 yapı şeması: main ve girdi, hesap, çıktı fonksiyonları



Gereksinimler. (1) Kurallar değişmez (RTEÜ tablosu, %40 + %60, final barajı 50, şartlı DC; Recep Tayyip Erdoğan Üniversitesi, 2025). (2) Fonksiyonlar üç gruptadır: doğrulamalı girdi, yan etkisiz hesap ve çıktı (Şekil 8.9). (3) `-DOBS_TEST` ile `main` menü yerine `testleri_calistir()`'ı çalıştırır (37 `assert`). (4) Aynı girdilerle v0.7'den tek fark başlık satırıdır.

Hesap fonksiyonları kısadır; örneğin `gecti_mi` tek satırdır: `return final >= FINAL_BARAJI && yuvarlanmis >= GECME_ORTALAMA;`. Yazmaz, okumaz; bu yüzden kolay test edilir. `harf_notu` ise `const char *` döndürür: "değiştirilemeyen bir metnin adresi"; şimdilik yalnızca `printf("%s", ...)` ile yazdırıyoruz (H12–H13). v0.7'de üç kez yazılan doğrulama tek bir `not_oku(etiket)` oldu.

`-DOBS_TEST` ile derlenen program "Tüm testler geçti (37 doğrulama)." yazar. Testler her harf aralığının **iki ucunu** ve barajı dener. Bilinçli bozulmuş kopyada (`testler/mini_obs_v0_8_bozuk_kural.c`) BA sınırı `> 81` yapıldı: menüyle bu hata ancak tam 81 alan bir öğrenciyle fark edilirdi; test ilk çalıştırmada, satır numarasıyla yakaladı (gerçek çıktı): "mini_obs_v0_8_bozuk_kural.c:350: testleri_calistir: Assertion 'strcmp(harf_notu(81, 81), "BA") == 0' failed.".

Tablo 8.8
Mini-OBS v0.8 test senaryoları (girdi dosyasıyla gerçek çalıştırma; son özet satırları)

Test	Senaryo	Sınıf ort.	Başarılı	Şartlı	Çıkış	v0.7 ile diff
T1	5 öğrenci, araya istatistik; sonunda özet ve çıkış	66.48	3 / 5	1 / 5	0	yalnızca başlık
T2	geçersiz menü seçimleri ve not/numara girdileri	76.97	1 / 2	0 / 2	0	yalnızca başlık
T3	kayıt yokken istatistik, sonra çıkış	—	—	—	0	yalnızca başlık
T4	sınır değerler: final tam 50; 59.5 → 60 (CC); 49.6 → 50 (DC); 89.5 → 90 (AA)	64.65	3 / 4	1 / 4	0	yalnızca başlık
T5	not girişinin ortasında girdi biter (EOF)	—	—	—	1	yalnızca başlık

Kabul ölçütleri. Sıfır uyarı (normal ve `-DOBS_TEST`); 37 doğrulama geçer; T1–T5'te v0.7 ile `diff` yalnızca başlığı gösterir, çıkış kodları aynıdır; hesap fonksiyonları `printf`/`scanf` çağırmaz. **Bilinçli sınır:** `istatistik_yaz`'ın sekiz parametresi var (istatistikler `main`'de yaşıyor); H9'da (v0.9) modül içi `static` durum ve `obs.h` / `obs.c` / `main.c` ile sadeleşecek.

8. Alıştırmalar ve Atölye

Atölye. (1) **Yeniden düzenleme** (25 dk, eşli): H5–H7'den bir programınızı fonksiyonlara bölün; önce yapı şemasını çizin, sonra iki sürümün çıktısını `diff` ile karşılaştırın. (2) **Sözleşme yazma** (15 dk): `ters_cevir`, `us_al` için ön/son koşul yazın; eşinizin sözleşmesinden test türetin. (3) **assert ile test** (20 dk): her fonksiyona normal, sınır ve hatalı girdi testi.

Alıştırma 8.1 ★☆☆ *Çıktı tahmini*

Programın üç satırlık çıktısını çalıştırmadan tahmin edin (`H08_A01_cikti_tahmini.c`). `a` hangi değeri yazar, neden?

alistirmalar/H08_A01_cikti_tahmini.c · satır 10–24

```c
int main(void)
{
    int a = 5;
    int b = f(a);
    printf("%d %d\n", a, b);
    printf("%d\n", f(f(a)));
    printf("%d\n", f(a) + f(a));
    return 0;
}

int f(int x)
{
    x = x * 2;
    return x + 1;
}
```

Alıştırma 8.2 ★★☆ *Çıktı tahmini*

Çağrılar hangi sırayla çalışır (`H08_A02_cagri_sirasi.c`)? Ekrana yazılanları ve `r`'nin değerini bulun. Prototip olmadığı hâlde program neden derlenir?

alistirmalar/H08_A02_cagri_sirasi.c · satır 9–26

```c
int g(int n)
{
    printf("g(%d) ", n);
    return n + 1;
}

int f(int n)
{
    printf("f(%d) ", n);
    return g(n * 2);
}

int main(void)
{
    int r = f(g(1));
    printf("\nsonuç: %d\n", r);
    return 0;
}
```

Alıştırma 8.3 ★★☆ *Hata bul*

Vize 72, final 85 için “Ortalama: 79.80” ve “Harf: B” yazması gereken programda **üç** hata var (`H08_A03_hata_bul_ortalama.c`). Program derlenmiyor: derleyici iletilerini okuyarak ikisini bulun; üçüncüsü düzeltilince görünür.

alistirmalar/H08_A03_hata_bul_ortalama.c · satır 9–31

```c
int main(void)
{
    int vize = 72, final = 85;
    printf("Ortalama: %.2f\n", ortalama(vize, final));
    printf("Harf    : %c\n", harf(ortalama(vize, final)));
    return 0;
}

double ortalama(int vize, int final)
{
    return (vize * 40 + final * 60) / 100;
}

char harf(double ort)
{
    if (ort >= 85) {
        return 'A';
    } else if (ort >= 70) {
        return 'B';
    } else if (ort >= 50) {
        return 'C';
    }
}
```

Alıştırma 8.4 ★★☆ *Hata bul*

Beklenen çıktı “E: ünlü” ve “K: ünlü değil”, ama program “e: ünlü” ve “k: ünlü” yazıyor (`H08_A04_hata_bul_harf.c`). İki hatayı bulun; hangisini derleyici uyarır?

alistirmalar/H08_A04_hata_bul_harf.c · satır 21–35

```c
void incele(char h)
{
    toupper(h);
    if (unlu_mu(h)) {
        printf("%c: ünlü\n", h);
    } else {
        printf("%c: ünlü değil\n", h);
    }
}

/* Son koşul: h büyük ünlü harf (A, E, I, O, U) ise true. */
bool unlu_mu(char h)
{
    return h == 'A' || 'E' || 'I' || 'O' || 'U';
}
```

Alıştırma 8.5 ★★☆ *Parsons*

Satırları, `us_al` fonksiyonunu tanımlayıp "3^4 = 81" yazdıran programa dönüşecek biçimde sıralayın ve girintileyin; bir satır fazladır (`H08_A05_parsons_us.c`).

alistirmalar/H08_A05_parsons_us.c · satır 8–22

```
    return sonuc;
long long us_al(int taban, int us);
int main(void)
}
    long long sonuc = 0;
{
        sonuc *= taban;
    printf("3^4 = %lld\n", us_al(3, 4));
long long us_al(int taban, int us)
    return 0;
    for (int i = 1; i <= us; i++) {
}
    long long sonuc = 1;
    }
{
```

Alıştırma 8.6 ★★☆ *Kod tamamla*

Boşlukları doldurun (`H08_A06_kod_tamamla_basamak.c`). Beklenen: "38783: 5 basamak, en büyük basamak 8" ve "0: 1 basamak, en büyük basamak 0".

alistirmalar/H08_A06_kod_tamamla_basamak.c · satır 20–42

```
/* Ön koşul: n >= 0.  Son koşul: n'nin basamak sayısı (0 için 1). */
int basamak_sayisi(int n)
{
    int sayac = 0;
    ____ {
        sayac++;
        n /= 10;
    } while (n != 0);
    return ____;
}

/* Ön koşul: n >= 0.  Son koşul: n'nin en büyük basamağı. */
____ en_buyuk_basamak(int n)
{
    int enb = n % 10;
    while (n > 0) {
        if (n % 10 > enb) {
            enb = ____;
        }
        n /= 10;
    }
    return enb;
}
```

Alıştırma 8.7 ★☆☆ *Kod yaz*

`int isaret(int x)` (−1, 0 ya da 1 döndürür) ve onu kullanan `int mutlak(int x)` fonksiyonlarını yazın. Test: −12 → −1 ve 12; 0 → 0 ve 0. `mutlak`'ın ön koşulu nedir? (İpucu: `INT_MIN`.)

Alıştırma 8.8 ★★☆ *Kod yaz*

`bool is_perfect(int n)`: n, kendisi hariç bölenlerinin toplamına eşitse mükemmel sayıdır (H7). Önce `int bolen_toplami(int n)` yazın. 10 000'e kadar mükemmel sayıları listeleyin (4 tane). Bölen aramayı `d <= n / d` ile nasıl hızlandırırsınız?

Alıştırma 8.9 ★★☆ *Kod yaz*

`int gun_sayisi(int ay, int yil)`: o aydaki gün sayısını döndürsün, geçersiz ayda −1; `is_leap_year`'ı kullansın. Sözleşmesini yazın ve `assert` ile en az dokuz test ekleyin: normal (Ocak, Nisan), sınır (Şubat 2024, 1900, 2000; Aralık) ve hatalı (ay 0, ay 13).

Alıştırma 8.10 ★★★ *Kod yaz*

Yeniden düzenleme. H6'daki `H06_13_sayi_tahmin.c`'yi `gizli_sayi_uret`, `tahmin_oku`, `ipucu_ver` ve `sonuc_yaz` fonksiyonlarına bölün. Önce yapı şemasını çizin. `-DSABIT_TOHUM=2026` ile eski ve yeni sürüm aynı girdilerde (`50 25 37 43 40 38`) aynı çıktıyı vermeli.

Alıştırma 8.11 ★★★ *Kod yaz*

Armstrong sayıları. k basamaklı n, basamaklarının k'inci kuvvetleri toplamına eşitse Armstrong sayısıdır ($153 = 1^3 + 5^3 + 3^3$). `basamak_sayisi`, `us_al`, `armstrong_toplami` ve `is_armstrong` fonksiyonlarının bileşimiyle 100 000'den küçük bütün Armstrong sayılarını yazdırın (20 tane, 0 dahil). Hangi fonksiyonları önceki örneklerden hazır aldınız?

9. Kendini Değerlendirme

- ☐ Prototip, tanım ve çağrıyı yazabiliyor; void ile (void) farkını açıklayabiliyorum.
- ☐ takas(a, b)'nin neden çalışmadığını bellek kutularıyla çizebiliyorum (değerle aktarım).
- ☐ Ön/son koşul yazabiliyor; bool döndüren doğrulama fonksiyonları ve bileşim kurabiliyorum.
- ☐ math.h, ctype.h, stdlib.h ve time.h fonksiyonlarını belgesinden okuyup kullanabiliyorum.
- ☐ Bir problemi adım adım iyileştirip yapı şemasını çizebiliyorum.
- ☐ assert ile normal, sınır ve hatalı girdi testi yazabiliyor; call/ret'i assembly'de gösterebiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · **Her algoritma bir fonksiyondur**

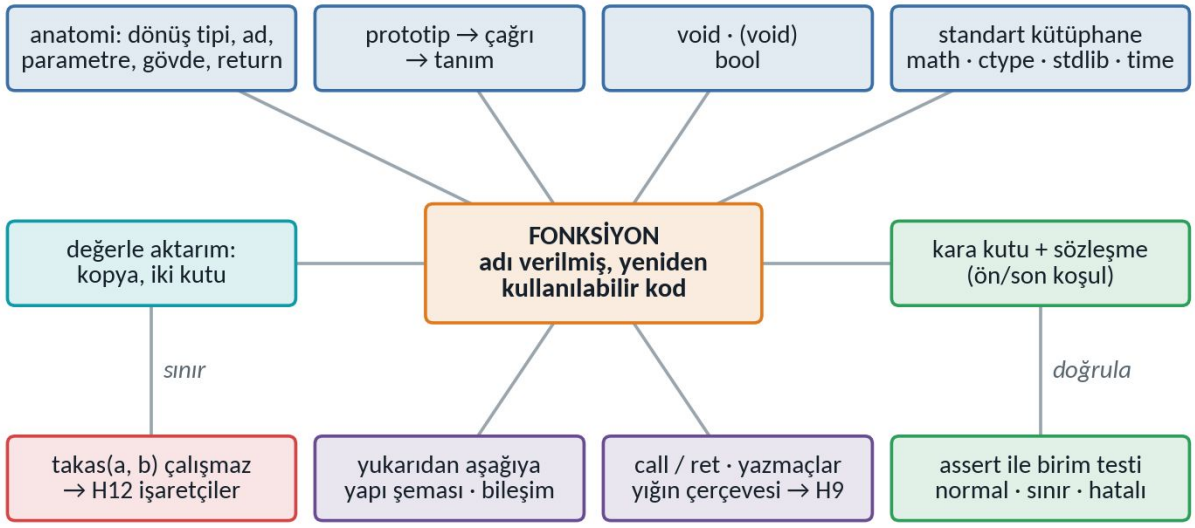
- AP-2'de her algoritma (arama, sıralama, EBOB, üs alma) test edilebilir bir **fonksiyondur**. Sözleşme ve `assert` **doğruluğu** göstermenin ilk adımıdır; aynı sözleşmenin iki gerçekleştirimini (K02: döngü ve formül) karşılaştırmak verimlilik tartışmasının biçimidir.
- Sıralamanın temel işlemi **takastır**; Örnek 10'daki başarısız `takas(a, b)` swap'ın işaretçi gerektirdiğini gösterir (H12; H10'da dizi elemanlarına uygulanacak).

> MERAK KUTUSU · **Küçük fonksiyonun bedeli var mı?**
>
> Her çağrı bir `call`, bir `ret` ve çerçeve kurulumudur; ama `-O2` ile derleyici `topla`'yı satır içine aldı ve hiç çağırmadı (Bölüm 3.10). Ölçmeden "fonksiyon yavaştır" demeyin.

11. Özet, Kavram Haritası ve Terim Sözlüğü

Özet. Fonksiyon = dönüş tipi + ad + parametre listesi + gövde; `return` değeri verir ve fonksiyonu bitirir; prototip imzayı önceden bildirir. Argümanlar **değerle** aktarılır (`takas` bu yüzden çalışmaz). Sözleşme fonksiyonu kara kutu yapar, `assert` onu sınar. Problemler yukarıdan aşağıya iyileştirilir, yapı şemasıyla çizilir. Makinede çağrı = `call` + `ret`; argümanlar (x86-64 Linux) yazmaçlarla gider.

Şekil 8.10
Haftanın kavram haritası



Büyük Sorunun yanıtı. Milyonlarca satırlık yazılımlar nasıl geliştirilir ve doğrulanır? Kimse milyonlarca satırı aklında tutmaz: yazılım, adı ve sözleşmesi tanımlı **parçalara** bölünür; bir parçayı yazan, diğerlerinin yalnızca sözleşmelerini bilir (soyutlama). Her parça kendi testleriyle doğrulanır, birleşince testler yeniden çalışır (Apollo 11'de öncelikli iş parçaları aşırı yükte en önemli işi sürdürdü). Biz de ATM'yi test edilebilir fonksiyonlara böldük ve çıktının değişmediğini `diff` ile **kanıtladık**.

Terim sözlüğü

Tablo 8.9
Haftanın terimleri (yol haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Kısa açıklama
fonksiyon; çağrı	function; call	Adı verilmiş kod bloğu; onu çalıştırma
parametre; argüman	parameter; argument	Tanımdaki değişken; çağrıdaki değer
dönüş değeri; dönüş tipi	return value; return type	return ile verilen değer; onun tipi
prototip (bildirim); tanım	prototype (declaration); definition	İmzanın önceden bildirimi; gövdeli fonksiyon
değerle aktarım	pass by value	Argümanın kopyasının parametreye verilmesi
sözleşme; ön koşul; son koşul	contract; precondition; postcondition	Çağıranın ve fonksiyonun yükümlülükleri
adım adım iyileştirme; yapı şeması	stepwise refinement; structure chart	Yukarıdan aşağıya tasarım; çağrı hiyerarşisi

Türkçe	İngilizce	Kısa açıklama
birim testi; doğrulama (assert)	unit test; assertion	Bir fonksiyonu bilinen girdilerle sınamak
yeniden düzenleme	refactoring	Davranışı koruyarak yapıyı iyileştirmek
çağrı kuralı; dönüş adresi	calling convention (ABI); return address	Argümanların yeri; ret'in döneceği adres

Gelecek hafta: Fonksiyonlar II. İç içe çağrılardan sonra bilgisayar kimin nereye döneceğini nasıl hatırlar? Bir fonksiyon **kendini** çağırırsa ne olur (özyineleme, Hanoi Kuleleri)? Ayrıca kapsam, ömür (`static`), çok dosyalı projeler ve Mini-OBS v0.9. Hazırlık: Python Tutor'da `H08_K06_cagri_akisi.c`'yi izleyin.

12. Kaynaklar

Adler, P. (1998). *Apollo 11 program alarms*. In E. M. Jones & K. Glover (Eds.), *Apollo lunar surface journal*. NASA. https://www.nasa.gov/wp-content/uploads/static/history/alsj/a11/a11.1201-pa.html

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Computer History Museum. (t.y.). *Margaret Hamilton* [Fellow profile]. https://computerhistory.org/profile/margaret-hamilton/

GCC Team. (2024). *Porting to GCC 14*. Free Software Foundation. https://gcc.gnu.org/gcc-14/porting_to.html

Guo, P. J. (2013). Online Python Tutor: Embeddable web-based program visualization for CS education. In *Proceedings of the 44th ACM Technical Symposium on Computer Science Education* (pp. 579–584). ACM. https://doi.org/10.1145/2445196.2445368

Guo, P. J. (2021). Ten million users and ten years later: Python Tutor's design guidelines for building scalable and sustainable research software in academia. In *The 34th Annual ACM Symposium on User Interface Software and Technology (UIST '21)* (pp. 1235–1251). ACM. https://doi.org/10.1145/3472749.3474819

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Jones, E. M., & Glover, K. (Eds.). (t.y.). *Apollo 11 lunar surface journal: The first lunar landing*. NASA. https://www.nasa.gov/wp-content/uploads/static/history/alsj/a11/a11.landing.html

LLVM Project. (2023). *Clang 16.0.0 release notes*. https://releases.llvm.org/16.0.0/tools/clang/docs/ReleaseNotes.html

Martin, F. H. (1994). *Apollo 11: 25 years later*. In E. M. Jones & K. Glover (Eds.), *Apollo lunar surface journal*. NASA. https://www.nasa.gov/wp-content/uploads/static/history/alsj/a11/a11.1201-fm.html

Microsoft. (t.y.). *x64 calling convention*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention

NASA Science. (2024, 5 Kasım). *Margaret Hamilton*. https://science.nasa.gov/people/margaret-hamilton/

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

Wirth, N. (1971). Program development by stepwise refinement. *Communications of the ACM*, *14*(4), 221–227. https://doi.org/10.1145/362575.362577